Decision trees

Forest CoverType · seven species from a contour map

LECTURE 6 | GÉRON CH. 5

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

The models fitted so far predict with a *weighted sum*: $\boldsymbol{\theta}^{\mathsf{T}}\mathbf{x}$, fitted by solving a linear system or by descending a convex surface.

- They need their features scaled, because the geometry of the loss depends on the units
- They extend to curves only by manufacturing the curve yourself, as polynomial features
- Their explanation of a prediction is a sum of n terms — a reason, but not a short one

> **THE STANDING CONSTRAINT, UNCHANGED**
>
> Split before anything is fitted. All preprocessing inside a `Pipeline` passed to cross-validation. Nothing derived from the test set in the training path. Fixed seeds. Report the fold scores, not only the mean.

Lecture 3 reached for a random forest once, to draw a better precision–recall curve, and did not say what one was. Today's model is the thing a forest is made of: not a weighted sum at all, but a sequence of yes/no questions — and the sequence *is* the explanation.

Today

1. The task, the data, and what makes a tree the right family (15 min)
2. **The mathematics** — impurity: Gini, entropy, and the greedy CART objective they feed (20 min)
3. **The method** — growing a tree, reading it, tracing a prediction, and the hyperparameters that constrain it (40 min)
4. Regression trees, and two things that break a tree (15 min)

The fourth block is the one to stay awake for. Both failure conditions are properties of the algorithm rather than accidents, and the second of them is the whole reason the next lecture exists.

FIRST FIFTEEN MINUTES

The task, and why the model family is chosen first

15 minutes

The setting

An agency that manages a national forest has to publish a map of **forest cover type** — which species dominates each 30 by 30 metre patch of ground — over an area far too large to survey on foot.

> **THE TASK**
>
> From cartographic measurements alone — elevation, slope, aspect, distances to water, road and fire ignition points, hillshade, soil type — predict which of **seven** tree species covers the patch.

No satellite imagery, no remote sensing. Numbers a surveyor could in principle read off a contour map.

What the output feeds

The map is not an internal artefact. It goes into the public record, and it is used to decide:

- where logging is permitted and where it is not
- which parcels qualify for habitat protection
- how fire risk is modelled for the whole district

✗ A landowner who is refused a permit is entitled to ask *why*. So is a court.

The requirement that decides the model

> **STATED BY THE AGENCY, BEFORE ANY DATA IS SEEN**
>
> Every individual prediction must be accompanied by a **human-readable justification**: a statement, in terms of the measured quantities, of why *this* patch was classified as *that* species.

Note the word *individual*. Not “which variables matter in general”. Not “the model is 88% accurate”. This patch, this answer, this reason.

Requirements of this shape are ordinary in regulated work — credit, insurance, medicine, planning. They are worth meeting deliberately rather than discovering late.

Turn the requirement into a specification

“Human-readable” is not a testable condition. Before writing any code, negotiate it into one. We asked, and this is what came back:

Requirement	Testable form
Readable by a non-specialist	stated as conditions on the measured columns, not on transformed ones
Short enough to check	at most 8 conditions per prediction
Auditable	the same patch gives the same justification, and the rule can be applied by hand

✓ Eight is the number the agency chose. It is arbitrary — and it is now a hyperparameter fixed by the brief rather than by cross-validation. That is unusual and worth noticing.

What that rules out, before we start

Model	Can it justify one prediction?
Logistic regression	**partly — a weighted sum of 54 terms is a reason, but not a short one ✓**
k-nearest neighbours	**“because these 5 other patches looked similar” ✗**
SVM with an RBF kernel	**no — the decision surface has no finite description ✗**
Neural network	**no ✗**
Decision tree	**yes — the path from root to leaf *is* the justification ✓**

This is the first problem in the course where the model family is chosen by the **shape of the answer** rather than by the data.

Frame the problem

Question	Answer	Because
Supervision	✓ **supervised**	every surveyed patch carries its species
Task	✓ **multiclass classification**	seven mutually exclusive cover types — not multilabel, not multioutput
Learning mode	✓ **batch, offline**	the survey is finished; nothing streams in
Assumptions	✓ **two, stated**	the classes are exhaustive and exclusive; the survey represents the mapped area

WHEN THE FOURTH ROW BITES

If the agency wants a *probability* per species rather than a label, both the metric and the model change: a tree's probabilities are the class proportions of one leaf, shared by every patch reaching it. We asked. They want a label, with a reason.

The data

The **Forest CoverType** survey: 581,012 patches of 30 by 30 metres in the Roosevelt National Forest, Colorado, each with 54 cartographic measurements and a surveyed cover type.

Columns	How many	Kind
Elevation, aspect, slope, hillshade, distances	10	quantitative
Wilderness area	4	already one-hot
Soil type	40	already one-hot

No missing values, no text columns, nothing to impute. This one arrives clean — so the difficulty is entirely elsewhere.

Fetching it

```
from sklearn.datasets import fetch_covtype

cover = fetch_covtype(as_frame=True)     # ~11 MB, cached after the first call
X_all, y_all = cover.data, cover.target

print(X_all.shape, y_all.nunique())      # (581012, 54) 7
```

`fetch_covtype` downloads once into `~/scikit_learn_data` and reads from disk afterwards. The classes are labelled 1 to 7, not 0 to 6 — a default nobody asked for, and one that will bite an index somewhere if you forget it.

We are going to use a tenth of it

```
X, _, y, _ = train_test_split(X_all, y_all, train_size=60_000,
                              stratify=y_all, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

60,000 patches out of 581,012, stratified so the class proportions are preserved exactly, then split 48,000 / 12,000.

> **SAY WHAT YOU DID, AND WHY**
>
> A 200-tree ensemble on all 581,012 rows takes minutes per fit, which does not fit inside a ninety-minute session. Every number in this lecture and the next is measured on the subsample, and every one of them would move if you used the full set.

Subsampling for speed is a legitimate choice and a *reportable* one. Subsampling and not mentioning it is not.

Split before you look — still

The rule has not changed, and neither has the reason.

```
assert len(X_train) == 48_000 and len(X_test) == 12_000
assert set(X_train.index).isdisjoint(X_test.index)
assert X_train.isna().sum().sum() == 0
```

Three assertions, three seconds, and the shape of the next ninety minutes is now guaranteed rather than hoped for.

Stratifying on the label matters here: with one class at 0.5% of the data, an unstratified split can hand a fold almost none of it.

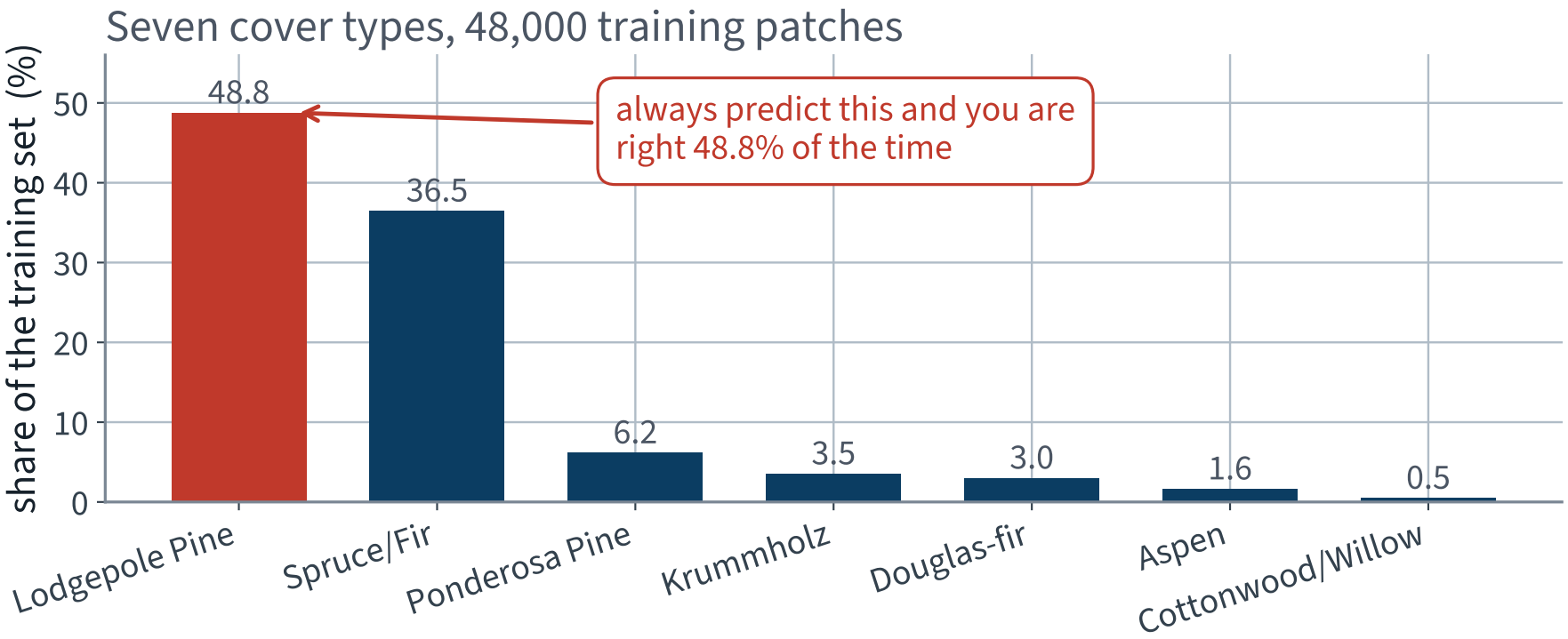
Look at the labels first

```
>>> y_train.value_counts()
2    23405     Lodgepole Pine
1    17501     Spruce/Fir
3     2954     Ponderosa Pine
7     1694     Krummholz
6     1435     Douglas-fir
5      784     Aspen
4      227     Cottonwood/Willow
```

✗ The commonest class is a hundred times the rarest.

You met this shape in Lecture 3, on a rare-event detector, where a classifier that never fired scored 90.96%. It did not go well then either.

Seven classes, and two of them are most of the forest



✗ Two species are 85% of the ground. Whatever we build has to be judged against the model that always says “Lodgepole Pine”.

Metric before model, and the anchor beside it

Seven classes, no natural ordering, no asymmetric cost stated by the agency. The default is **accuracy** — reported, as always, beside the number that says what *nothing* scores.

```
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print(f"{dummy.score(X_test, y_test):.1%}")     # 48.8%
```

Model	Test accuracy	Species it can ever predict
Always “Lodgepole Pine”	48.8%	✗ 1 of 7

✓ Anything below 48.8% is worse than a constant. The distance above it is the only part you actually earned.

What number would count as success?

- The constant model scores **48.8%**
- The agency's surveyors are stated to agree with one another on roughly nine patches in ten — an *assumption we were handed*, not a measurement we made
- So the useful range lies between those two, and the eight-condition requirement may well stop us short of the top of it

> A model that is 5 points worse and can be defended in a hearing may be worth more to this agency than one that is 5 points better and cannot. That is not a machine learning judgement, and it is not yours to make alone.

Keep both numbers in mind. We will measure the price of the constraint later in the lecture and it is larger than most people guess.

THE MATHEMATICS

Impurity, and the objective CART minimises

Géron §5.2–5.4 — today we grow a tree, so today we should know what growing one

20 minutes · examinable

The question

You called `DecisionTreeClassifier().fit()`.

It chose one feature and one threshold at every node. Chose them to *minimise what*?

> **WHY YOU NEED THE ANSWER**
>
> Scikit-Learn offers two answers — `criterion="gini"` and `criterion="entropy"`. Deciding whether that switch is worth touching requires knowing what the two quantities are and how they differ, and the answer also explains why a tree ignores a rare class.

Notation, fixed for the whole derivation

- A **node** holds a subset of the training instances. Write m for how many
- K — the number of classes. Here $K = 7$
- p_k — the fraction of the node's instances in class k, so $p_k \geq 0$ and $\sum_{k=1}^{K} p_k = 1$
- $\mathbf{p} = (p_1, \ldots, p_K)$ — the node's *class distribution*. It is the only thing an impurity measure sees

A node is **pure** when some $p_k = 1$. It is **uniform** when every $p_k = 1/K$.

No probability theory beyond a finite distribution, and no calculus beyond a second derivative.

Step 1 · What an impurity measure has to do

Before choosing a formula, say what any acceptable one must satisfy. Two conditions, and they are the whole specification:

$$\text{(i)}\quad I(\mathbf{p}) = 0 \iff \mathbf{p} \text{ is pure}$$

$$\text{(ii)}\quad I \text{ is maximal at } \mathbf{p} = (1/K, \ldots, 1/K)$$

Condition (i) says a node we have finished with scores zero. Condition (ii) says the node we know least about scores worst.

Everything else — the choice between Gini and entropy — is a choice *within* this specification, not a disagreement about it.

Step 2 · The general form

Both of Scikit-Learn's criteria are of one shape: sum a single function of each class's share.

$$I(\mathbf{p}) = \sum_{k=1}^{K} \varphi(p_k)$$

with $\varphi : [0,1] \to \mathbb{R}$ **strictly concave** and $\varphi(0) = \varphi(1) = 0$.

Concavity is not decoration. It is the property that delivers both conditions of Step 1 *and* the guarantee in Step 7, and it is the only property either proof uses.

$\varphi(0) = \varphi(1) = 0$ says a class that is absent and a class that is everything both contribute nothing to disorder.

Step 3 · Gini impurity

Take $\varphi(p) = p(1-p)$:

$$G(\mathbf{p}) \;=\; \sum_{k=1}^{K} p_k(1-p_k) \;=\; 1 - \sum_{k=1}^{K} p_k^2$$

GINI IMPURITY — SCIKIT-LEARN'S DEFAULT

WHAT IT MEASURES

Draw two of the node's instances independently, with replacement. $\sum_k p_k^2$ is the probability that they are the *same* class, so G is the probability that they are **different**. A pure node never disagrees with itself.

Step 4 · Entropy

Take $\varphi(p) = -p \log_2 p$, with $\varphi(0) = 0$ by the limit $p \log p \to 0$:

$$H(\mathbf{p}) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

SHANNON ENTROPY, IN BITS

WHAT IT MEASURES

The average number of **bits** needed to transmit the class of an instance drawn from the node, under the best possible code. A pure node needs none, because the receiver already knows the answer.

Step 5 · Both formulas satisfy Step 1

(i) $\varphi \geq 0$ on $[0,1]$ and $\varphi(p) = 0$ only at $p \in \{0,1\}$, for both choices. So $I(\mathbf{p}) = 0$ forces every $p_k \in \{0,1\}$, and since they sum to 1 exactly one is 1: the node is pure.

(ii) By Jensen's inequality on the concave φ,

$$\frac{1}{K}\sum_k \varphi(p_k) \;\leq\; \varphi\left(\frac{1}{K}\sum_k p_k\right) = \varphi\left(\tfrac{1}{K}\right)$$

so $I(\mathbf{p}) \leq K\,\varphi(1/K)$.

Strict concavity makes the inequality an equality only when every p_k is equal. The maximum is attained at the uniform distribution and nowhere else.

The two maxima, evaluated

$$G_{\max} = K \cdot \frac{1}{K}\left(1 - \frac{1}{K}\right) = 1 - \frac{1}{K}$$

$$H_{\max} = K \cdot \left(-\frac{1}{K}\log_2 \frac{1}{K}\right) = \log_2 K$$

K	$G_{\max}$	$H_{\max}$ (bits)
2	0.5	1
7	6/7 ≈ 0.857	$\log_2 7 \approx 2.807$

✗ The two are on different scales. Any comparison of their shapes has to divide one of them first, and saying which is part of the comparison.

Step 6 · The objective CART minimises

A split sends m_{L} instances left and m_{R} right, $m_{\mathrm{L}} + m_{\mathrm{R}} = m$. Its cost is the impurity of the two children, weighted by their size:

$$J(k, t_k) \;=\; \frac{m_{\mathrm{L}}}{m}\, I_{\mathrm{L}} \;+\; \frac{m_{\mathrm{R}}}{m}\, I_{\mathrm{R}}$$

SEARCHED OVER EVERY FEATURE K AND EVERY THRESHOLD T_K

Equivalently, maximise the **impurity decrease** $\Delta I = I_{\mathrm{parent}} - J(k, t_k)$. The parent term is the same for every candidate at this node, so the two are the same search.

Step 7 · A split can never increase impurity

Every instance goes left or right, so for each class the parent share is the size-weighted mixture of the children's shares:

$$p_k = \frac{m_{\mathrm{L}}}{m} p_{\mathrm{L},k} + \frac{m_{\mathrm{R}}}{m} p_{\mathrm{R},k} \qquad \text{for every } k$$

Apply concavity of φ to that convex combination and sum over k:

$$I(\mathbf{p}) \geq \frac{m_{\mathrm{L}}}{m} I_{\mathrm{L}} + \frac{m_{\mathrm{R}}}{m} I_{\mathrm{R}} = J \qquad \Longrightarrow \qquad \Delta I \geq 0$$

✓ So the greedy search can always stop safely: no split ever makes things worse, and $\Delta I = 0$ is the signal that this node has nothing left to gain.

Step 8 · Why the two criteria usually choose the same split

Both criteria rank candidate splits by ΔI, and ΔI depends only on the two child distributions and their weights.

> **THE REASON, IN ONE LINE**
>
> $I(\mathbf{p}) = \sum_k \varphi(p_k)$ with φ concave is **Schur-concave**: if one distribution *majorises* another — is more concentrated, in the sense that its largest share is larger, its two largest shares sum to more, and so on — then it has the lower impurity, *for every such* φ.

So take two candidate splits that send the same numbers left and right. If A's two children each majorise B's, then $J_{\mathrm{A}} \leq J_{\mathrm{B}}$ for *every* concave φ: Gini and entropy **must** agree that A is better. Neither the choice of φ nor its scale can change it.

… and where they are free to disagree

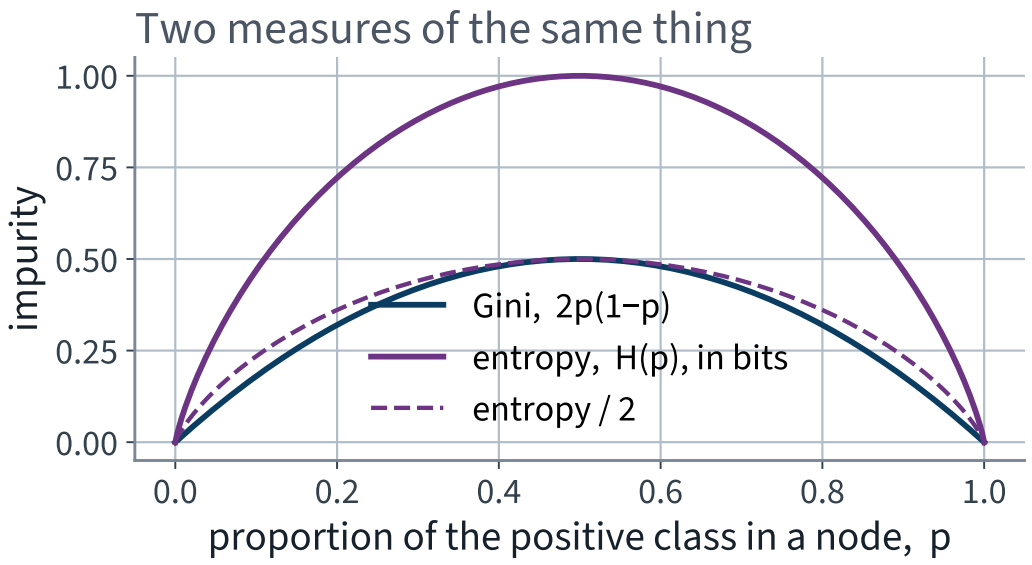
Only on two candidate splits whose child distributions are *not* comparable in that ordering. Then the ranking depends on the shape of φ. How far can they part? On a two-class node with $p_1 = p$,

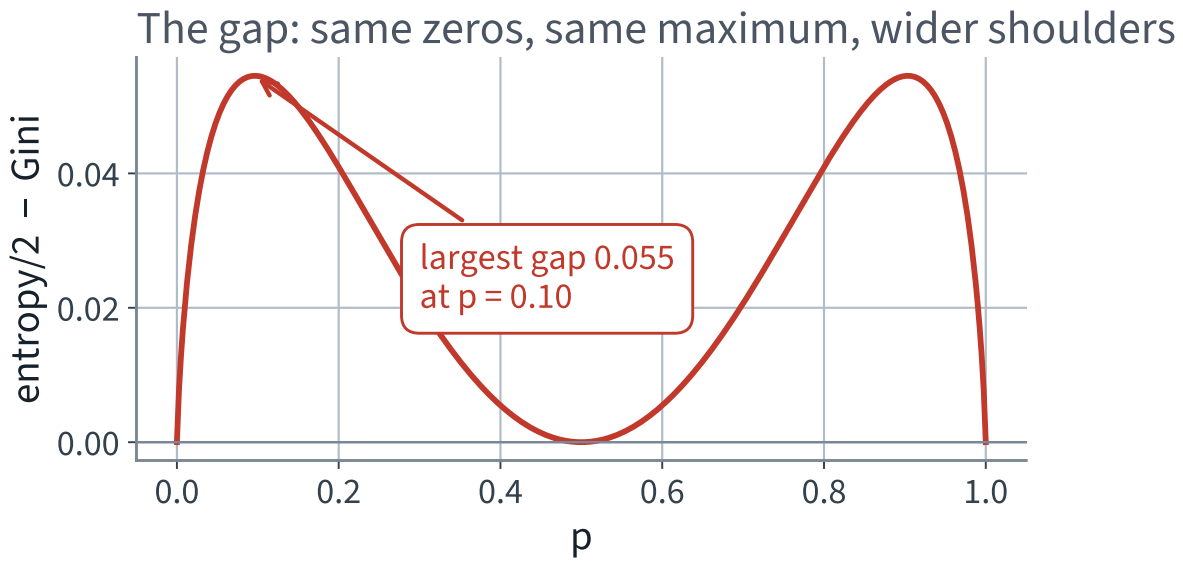
$$G(p) = 2p(1-p), \qquad H(p) = -p\log_2 p - (1-p)\log_2(1-$$

H has maximum 1 and G has maximum $1/2$, so compare $H/2$ with G. Both are then 0 at $p \in \{0, 1\}$ and $1/2$ at $p = 1/2$: they agree at three points, and the largest gap between them anywhere else is **0.055**, at $p \approx 0.10$ and again at $p \approx 0.90$.

✓ So two candidates whose weighted impurities differ by more than 0.055 are ranked identically by both criteria. Disagreement needs a close call *and* an incomparable pair.

Two measures of the same idea





Same zeros, same maximum, different shoulders. Entropy penalises a nearly-pure node more heavily than Gini does.

What that means in practice

- Entropy is steeper near the ends, so it is more reluctant to accept a node as “almost pure enough”
- Géron: entropy tends to produce slightly more *balanced* trees; Gini is marginally faster because there is no logarithm to evaluate
- “Most of the time it does not make a big difference”

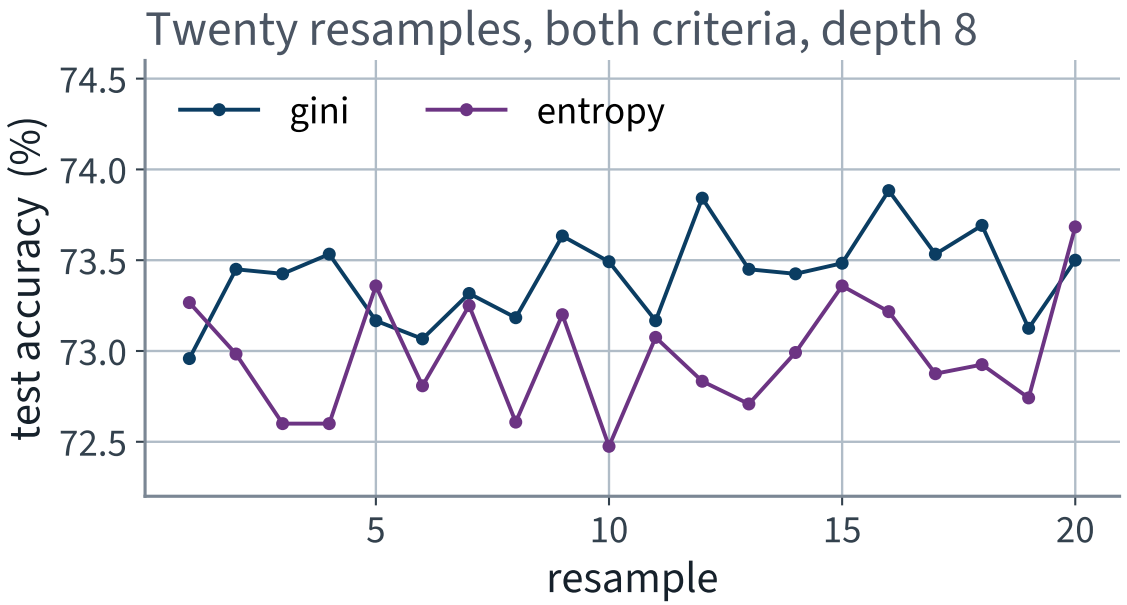
✗ That last sentence is a claim about behaviour. We have a dataset, a model and ninety minutes. Do not accept it — measure it.

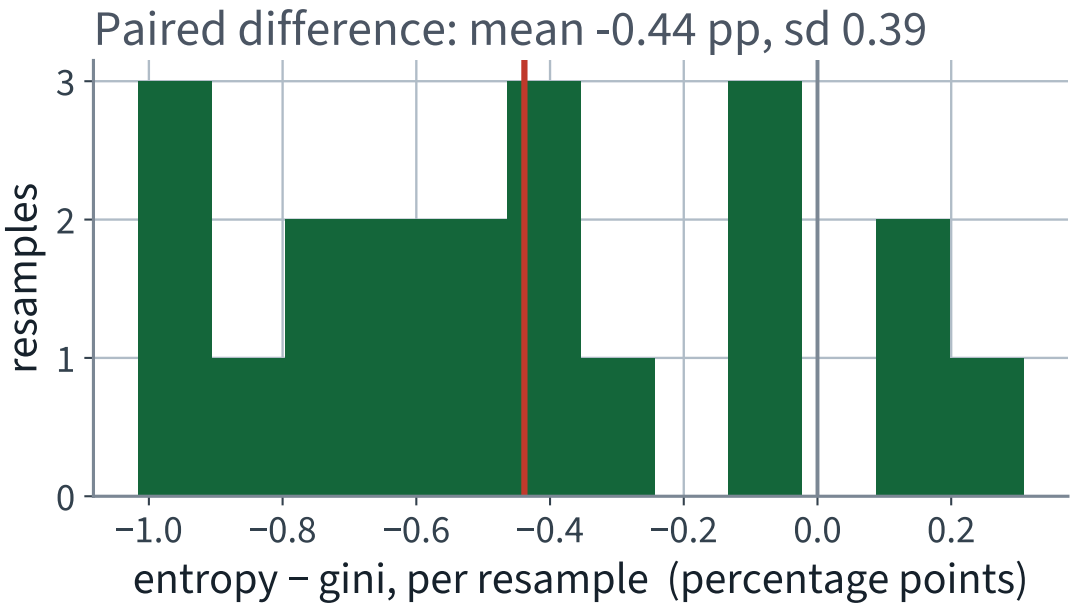
Measure it

```
for seed in range(20):
    Xs, _, ys, _ = train_test_split(X_train, y_train, train_size=0.8,
                                    stratify=y_train, random_state=seed)
    for crit in ("gini", "entropy"):
        t = DecisionTreeClassifier(criterion=crit, max_depth=8,
                                   random_state=42).fit(Xs, ys)
        # accuracy, the root split, and the 12,000 test predictions
```

Twenty resamples, both criteria on *the same* resample each time, so the comparison is **paired** and the resample-to-resample noise cancels.

Twenty resamples, both criteria





Gini is above entropy on 17 of the 20 resamples. The difference is small, and it is not noise.

The measured answer

Question	Measured
Resamples where the two chose the same root feature	**20 of 20 ✓**
Test predictions on which the two trees agree	85.3%
Mean accuracy, gini	73.4%
Mean accuracy, entropy	73.0%
Paired difference, entropy − gini	**✗ −0.44 ± 0.39 points**

Step 8 predicted this shape: the two criteria pick the same first question every time — the root split is not a close call — and then disagree about one prediction in seven, deep in the tree where candidates are separated by a hair.

Reading that honestly

Three statements, all true, and only the third is a recommendation:

- The effect is **real**: gini wins on 17 of the 20 resamples, and the mean sits about five standard errors from zero
- The effect is **tiny**: 0.44 points, against the 8.0 points that `max_depth` is worth later in this lecture
- So: **do not spend your tuning budget here.** Leave the default and go and change something that matters

> “Statistically detectable” and “worth acting on” are different claims and need different evidence. Twenty paired resamples give you both; one run gives you neither.

One thing the derivation does *not* give you

Step 7 guarantees each split is locally harmless. It says nothing about the **sequence** of splits, and CART chooses that greedily: the best split at this node, then never reconsidered.

> **WHEN IT BITES**
>
> A split that looks mediocre now but would enable two excellent splits below it is never found. Finding the optimal tree is **NP-complete**; the greedy algorithm returns a good one in $O(n\,m\log m)$ and no known algorithm returns the best one in reasonable time.

So the tree you get is a function of the algorithm as much as of the data. That is the seed of this lecture's second failure condition.

What that derivation buys you

EXAMINABLE

1. [1] Impurity is $\sum_k \varphi(p_k)$, φ strictly concave
   the only property the rest uses

2. [2] Gini: $\varphi(p) = p(1-p)$. Entropy: $\varphi(p) = -p \log_2 p$
   disagreement probability, and bits

3. [3] Zero exactly at purity, maximal exactly at uniformity
   Jensen, strictness giving uniqueness

4. [4] CART minimises $J = \frac{m_{\mathrm{L}}}{m} I_{\mathrm{L}} + \frac{m_{\mathrm{R}}}{m} I_{\mathrm{R}}$
   child impurities, weighted by size

5. [5] $\Delta I \geq 0$, and both criteria rank comparable splits alike
   concavity, then Schur-concavity

THE METHOD

Growing a tree, and reading one

40 minutes · examinable

What a decision tree is

A binary tree. Each internal node asks one question about one feature; each leaf carries a prediction.

- A question is always of the form $x_k \leq t_k$ — one feature, one threshold
- Prediction: start at the root, answer the question, go left or right, repeat until a leaf
- The leaf predicts the **majority class of the training instances that reached it** — the $\arg\max_k p_k$ of that node's class distribution

Operation	Cost	Why
Predict one instance	$O(\log_2 m)$	one root-to-leaf path of a roughly balanced tree
Train	$O(n \times m \log_2 m)$	at every node, sort and scan all n features

✓ Prediction does not depend on n, which is why reading a prediction costs the same as making it. That is what makes the justification free.

Fit one, with no constraints at all

```python
from sklearn.tree import DecisionTreeClassifier

free = DecisionTreeClassifier(random_state=42)      # no depth limit
free.fit(X_train, y_train)

print(f"depth          {free.get_depth()}")
print(f"leaves         {free.get_n_leaves():,}")
print(f"train accuracy {free.score(X_train, y_train):.1%}")
```

```
depth          33
leaves         5,699
train accuracy 100.0%
```

✗ 100.0%. A tree with 5,699 leaves and 48,000 training patches has eight patches per leaf on average. Of course it is perfect on its own data.

Measure it honestly

Unconstrained tree	Accuracy
On the training set	✗ 100.0%
5-fold cross-validated, on the training set	82.1%
On the test set (looked at once, at the end)	82.6%

82% is a real number and it is far above the 48.8% baseline, so the tree family works on this problem.

✗ But count what a prediction rests on. The tree is 33 levels deep, and the requirement allows eight conditions.

How long a justification actually is

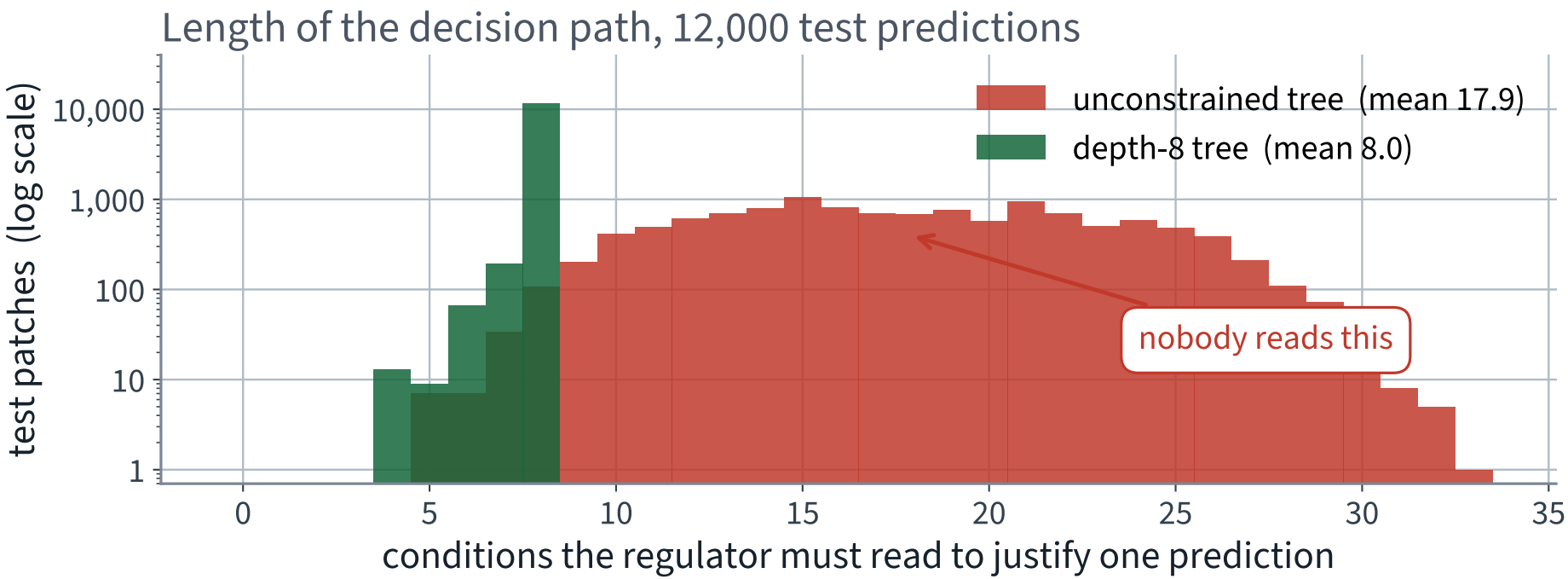
```
# decision_path marks every node visited, root and leaf included,
# so the number of conditions applied is that count minus one
path = free.decision_path(X_test)
lengths = np.asarray(path.sum(axis=1)).ravel() - 1

print(f"mean {lengths.mean():.2f}   max {lengths.max()}")
```

```
mean 17.88   max 33
```

✗ The brief allows eight conditions. The average justification this tree produces is 17.88, and the worst is 33.

The maximum equals `get_depth()`, which is the cheap check that the off-by-one went the right way.

Two models, the same 12,000 predictions



✗ One produces a reason a person can check; the other produces a paragraph nobody will read.

Why the tree grew to depth 33

Because nothing stopped it. A decision tree is a **nonparametric** model: the number of parameters is not fixed before training, so the structure is free to follow the data as far as the data goes.

> That freedom is what let it isolate almost every training patch. It is also what we now have to take away — here to meet the brief, and in general to stop it overfitting.

A linear model cannot do this. Its parameter count is decided by the number of columns before it sees a single row.

The regularisation hyperparameters

Parameter	What it controls
`max_depth`	hard ceiling on the length of every path — and therefore on the justification
`min_samples_split`	a node with fewer instances than this is never split
`min_samples_leaf`	a split that would leave a child smaller than this is rejected
`max_leaf_nodes`	ceiling on the total number of leaves — on the number of rules
`max_features`	how many features are considered at each split

Increasing the `min_*` parameters or decreasing the `max_*` ones regularises the model.

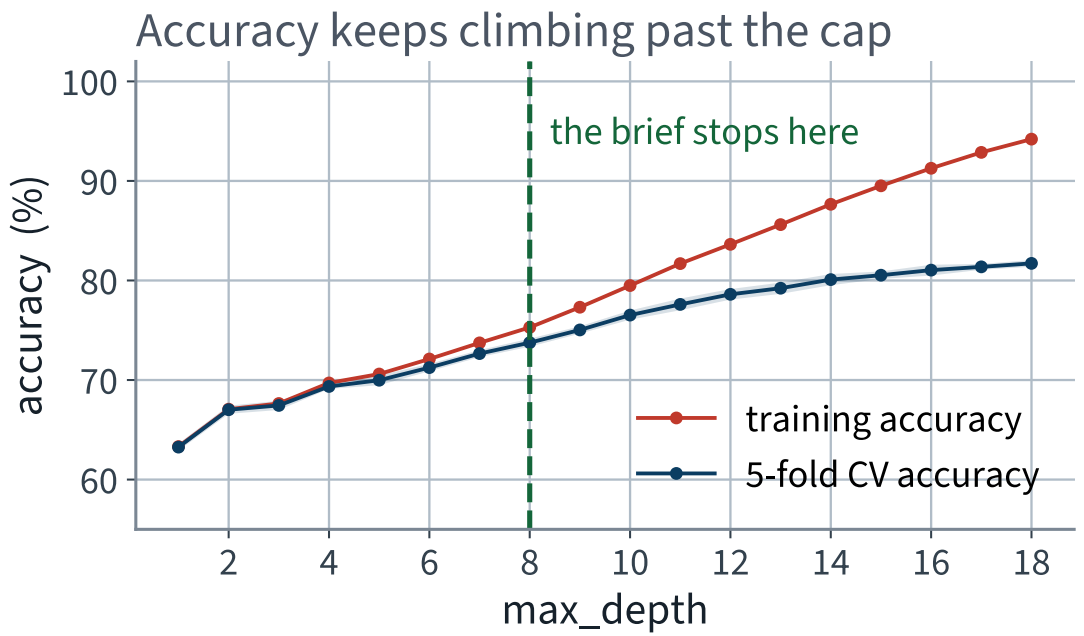
✓ Two of these control accuracy. One of them — `max_depth` — is fixed for us by the brief.

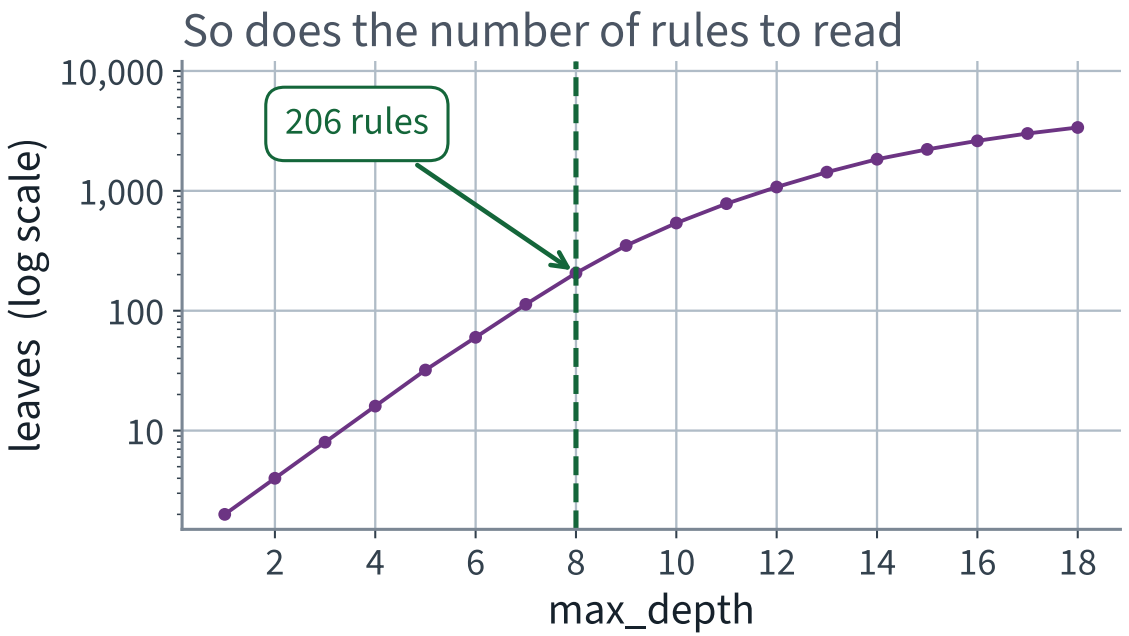
What does depth actually buy?

```
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for d in range(1, 19):
    clf = DecisionTreeClassifier(max_depth=d, random_state=42)
    scores = cross_validate(clf, X_train, y_train, cv=cv,
                            return_train_score=True, n_jobs=-1)
```

Cross-validated, on the training set. The test set is not consulted to choose anything — that rule survives from Lecture 2 and will survive to the last lecture.

Accuracy against depth, and the price of reading it





Cross-validated accuracy is still climbing at depth 18. Every point of it is bought with more rules than anyone will read.

Reading that curve

max_depth	CV accuracy	Leaves	What a justification looks like
1	63.3%	2	one condition
3	67.5%	8	a sentence
8 ✓	**73.8% ✓**	206	a short paragraph — the brief
18	81.7%	3,377	nobody reads this

Between depth 8 and depth 18 there are **8.0 points** of accuracy. That is the measured price of the eight-condition requirement, and it is a number worth putting in front of the agency.

✗ It is not, however, ours to spend. The requirement is the brief.

The one hyperparameter we may not tune

> **WHEN CROSS-VALIDATION ANSWERS THE WRONG QUESTION**
>
> Cross-validation says `max_depth=None`. The brief says `max_depth=8`. **Cross-validation does not get a vote on this one**, because it is not optimising the thing the agency is buying.

The first time in this course that the honest answer to “what does the search say?” is “the search is answering the wrong question”. It will not be the last.

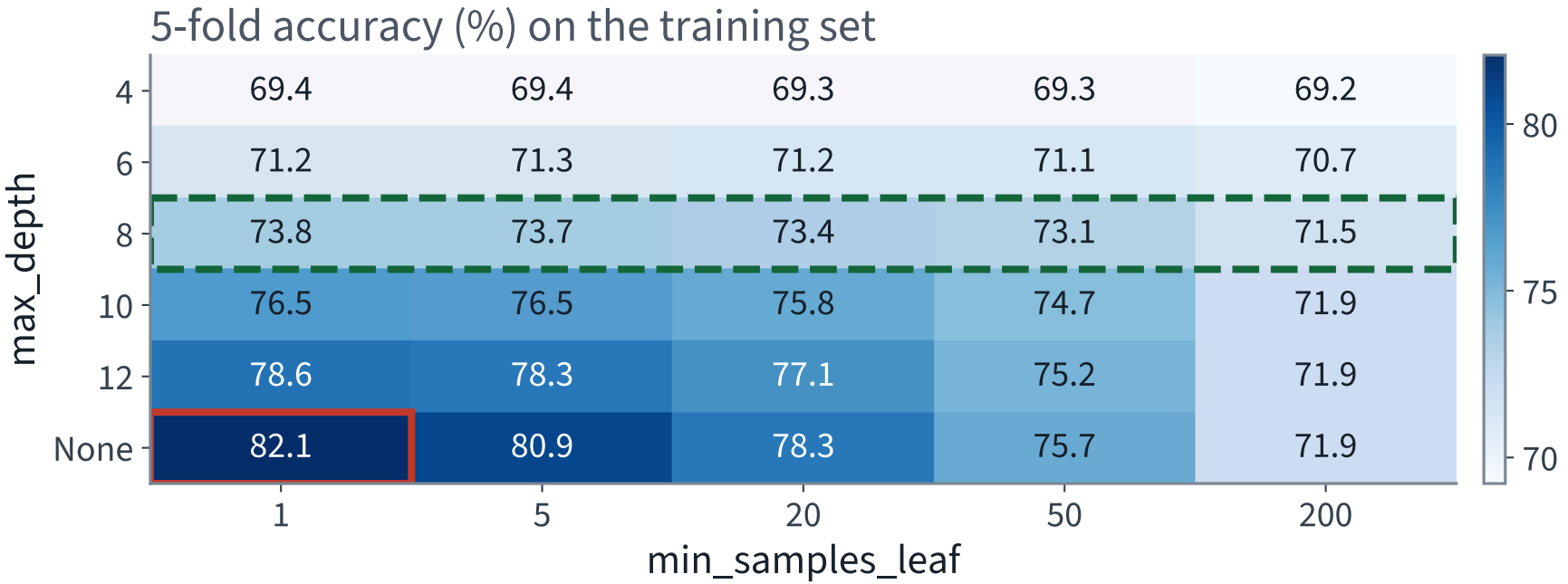
Bring the 8.0-point figure to the agency. Perhaps they will trade a condition or two for it. That is a conversation, not a `GridSearchCV`.

Tune what is left, inside the constraint

```
from sklearn.model_selection import GridSearchCV

grid = {"max_depth": [4, 6, 8, 10, 12, None],
        "min_samples_leaf": [1, 5, 20, 50, 200]}

search = GridSearchCV(DecisionTreeClassifier(random_state=42), grid,
                      cv=cv, n_jobs=-1).fit(X_train, y_train)
# 30 combinations x 5 folds = 150 fits
```

We search the whole grid anyway — including the depths we are not allowed to use — because the rows we cannot pick are exactly what tells the agency what its requirement costs.

Thirty combinations, and only one row is available to us



The red cell is the best model. The green row is the set of models we are allowed to ship.

Two things to read off that grid

- **Within the green row,** `min_samples_leaf` **barely matters** — 73.8% at 1, 71.5% at 200. At depth 8 the depth limit binds first, so the leaf-size limit has almost nothing left to do
- **Down the unlimited-depth row it matters enormously** — 82.1% at `min_samples_leaf=1` falling to 71.9% at 200. With no depth limit, the leaf size *is* the regularisation

> Two hyperparameters that both restrict the tree do not act independently. The 2-D grid shows that; two separate 1-D sweeps would not have.

So the grid's answer under the cap is `min_samples_leaf=1`. **We are not going to ship it** — next slide.

Why we ship `min_samples_leaf=20`

The model states its justification as “91% of the 463 training patches in this leaf”. At `min_samples_leaf=1` that becomes *“100% of the 1”* — a single surveyed patch wearing the grammar of evidence. The brief requires a justification that can be audited; that is not one.

	leaf	CV accuracy
what cross-validation prefers	1	73.75%
what the brief requires	**20 ✓**	73.35%
✓ **price of auditability**	—	**0.40 points**

✓ Same reason we overruled it on depth: when the brief constrains the model, the grid does not get a vote. And that 0.40 goes to the agency — a constraint that overrules a measurement has to show the measurement it overruled.

The model we are going to ship

```
tree = DecisionTreeClassifier(max_depth=8, min_samples_leaf=20,
                              random_state=42).fit(X_train, y_train)
```

	Unconstrained	Depth 8, leaf 20
Leaves	5,699	**163 ✓**
Columns consulted, of 54	47	**24 ✓**
Mean conditions per justification	**✗ 17.88**	**7.97 ✓**
Smallest leaf	**✗ 1**	**20 ✓**
Training accuracy	100.0%	74.4%
Cross-validated accuracy	82.1%	73.4%

Notice what the two accuracy rows now do

74.4% on the training set, 73.4% cross-validated. A gap of **one point**.

The unconstrained tree's gap was **17.9 points**. The depth limit did not only shorten the justification; it removed almost all of the overfitting as a side effect.

Which leaves a question worth holding on to: if the constrained tree barely overfits, why is it still nearly nine points worse than the unconstrained one? The answer is the subject of the last block, and of the next lecture.

Now read it

Two routes, and you want both. The classic one is `export_graphviz` to a `.dot` file; the portable one is `plot_tree`, which draws into a matplotlib axis.

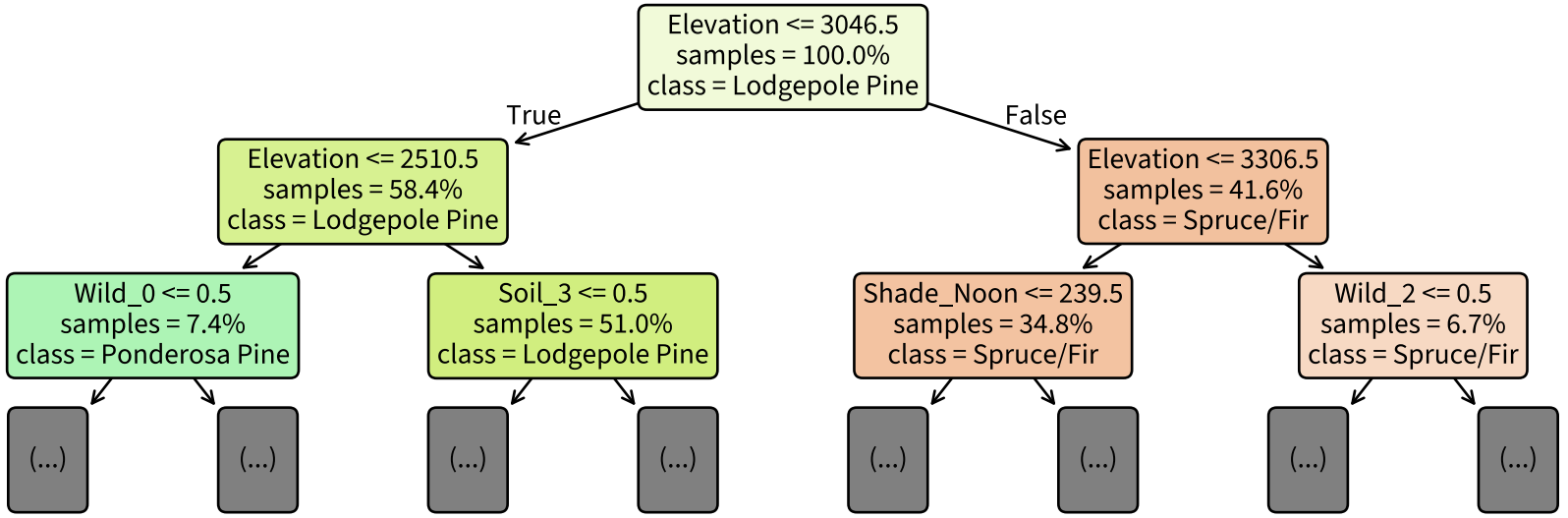
```
from sklearn.tree import export_graphviz, plot_tree

export_graphviz(tree, out_file="cover.dot", max_depth=2,
                feature_names=names, filled=True)
# then, in a terminal:  dot -Tpng cover.dot -o cover.png

plot_tree(tree, max_depth=2, feature_names=names,
          class_names=COVER_NAMES, filled=True, proportion=True, ax=ax)
```

`max_depth=2` in the drawing call is essential: all eight levels at a readable font size is about two metres of paper.

The `dot` binary is not a Python package and is not on a stock Colab runtime. The call succeeds, writes a file, and nothing renders.

NOT EXAMINABLE — ENGINEERING

The top three levels of the model



The first three questions the model asks about any patch in Colorado. Two of them are about elevation.

The same thing, as text

```
>>> print(export_text(tree, feature_names=names, max_depth=2, decimals=0))
|--- Elevation <= 3046
|   |--- Elevation <= 2510
|   |   |--- Wild_0 <= 0
|   |   |   |--- truncated branch of depth 6
|   |   |--- Wild_0 >  0
|   |   |   |--- truncated branch of depth 2
|   |--- Elevation >  2510
|   |   |--- Soil_3 <= 0
|   |   |   |--- truncated branch of depth 6
```

`export_text` needs no plotting library at all, and it is what you paste into an email. For a model whose selling point is that a person can read it, that matters.

Reading one node

Field	Meaning
`Elevation <= 3046.5`	the question; go left if true
`gini = 0.65`	Step 3's G at this node — 0 would be a single species
`samples = 48,000`	training patches that reached this node
`value = [0.36, 0.49, ...]`	their class distribution $\mathbf{p}$, seven entries
`class = Lodgepole Pine`	the majority — what a leaf here would predict

The threshold is 3,046.5 rather than 3,046 because CART puts the split **midway between two adjacent observed values**. Nothing in the data sits at 3,046.5 metres.

Trees need no feature scaling at all: $x_k \leq t_k$ is invariant under any strictly increasing transformation of x_k, so standardising a column changes its thresholds and nothing else.

Trace one prediction, all the way down

```
def justify(tree, x, names):
    t, node, out = tree.tree_, 0, []
    while t.children_left[node] != -1:
        f, thr = t.feature[node], t.threshold[node]
        left = x[f] <= thr
        out.append((names[f], "<=" if left else ">", thr, x[f]))
        node = t.children_left[node] if left else t.children_right[node]
    return out, node
```

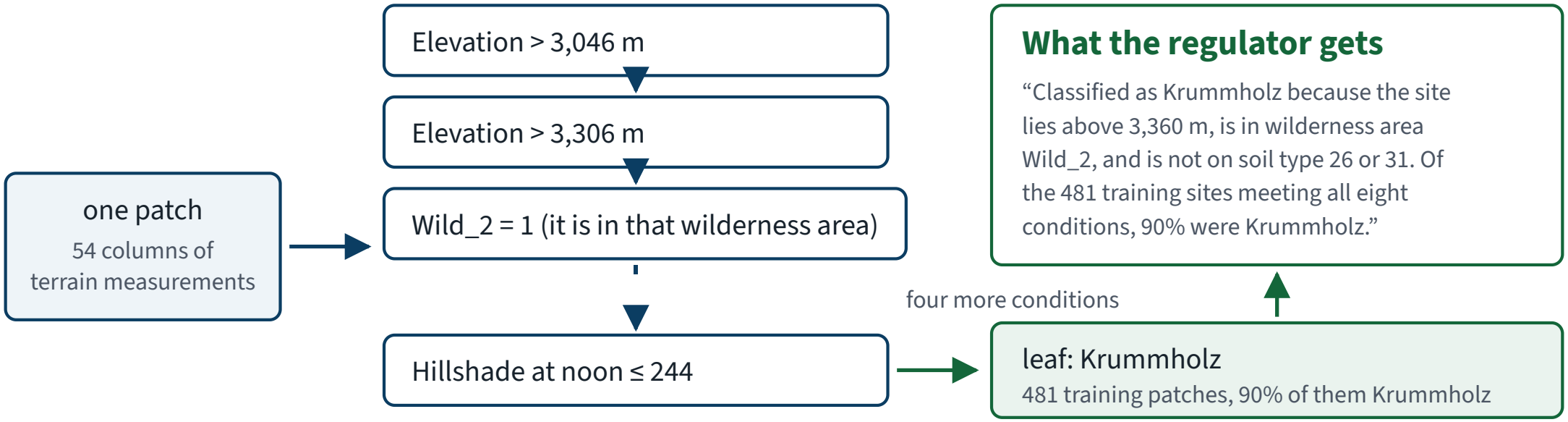
Nine lines, no library beyond what is already imported. The entire justification mechanism is `tree_.children_left`, `tree_.feature` and `tree_.threshold`.

One patch, eight conditions

```
>>> justify(tree, X_test.iloc[27].values, names)
1. Elevation   = 3,763  >  3,046
2. Elevation   = 3,763  >  3,306
3. Wild_2      = 1      >  0
4. Soil_31     = 0      <= 0
5. Elevation   = 3,763  >  3,360
6. HDist_Fire  = 3,020  >  364
7. Shade_Noon  = 200    <= 244
8. Shade_3pm   = 133    <= 196
-> Krummholz  (91% of the 463 training patches in this leaf)
```

✓ Eight conditions. Every one of them a measurement a surveyor took. The true species of that patch is Krummholz.

What the agency actually receives



✓ The path *is* the justification. No extra machinery, no approximation, no second model explaining the first.

Read the leaf carefully

91% of the 463 training patches in that leaf were Krummholz.

- That is a statement about **463 training patches**, not a probability that this patch is Krummholz
- `predict_proba` returns exactly those leaf proportions — so a tree's "probabilities" are piecewise constant and identical for every patch reaching the same leaf
- Across the 163 leaves of this tree the smallest holds 20 patches and the largest several thousand. Those deserve very different amounts of trust, and the count belongs in the justification

> **THE HONEST SENTENCE**
>
> "Of the training patches that satisfied these eight conditions, 91% were Krummholz." Not "this patch is 91% likely to be Krummholz."

The test set. Once.

73.0%

on 12,000 test patches the model has never seen.

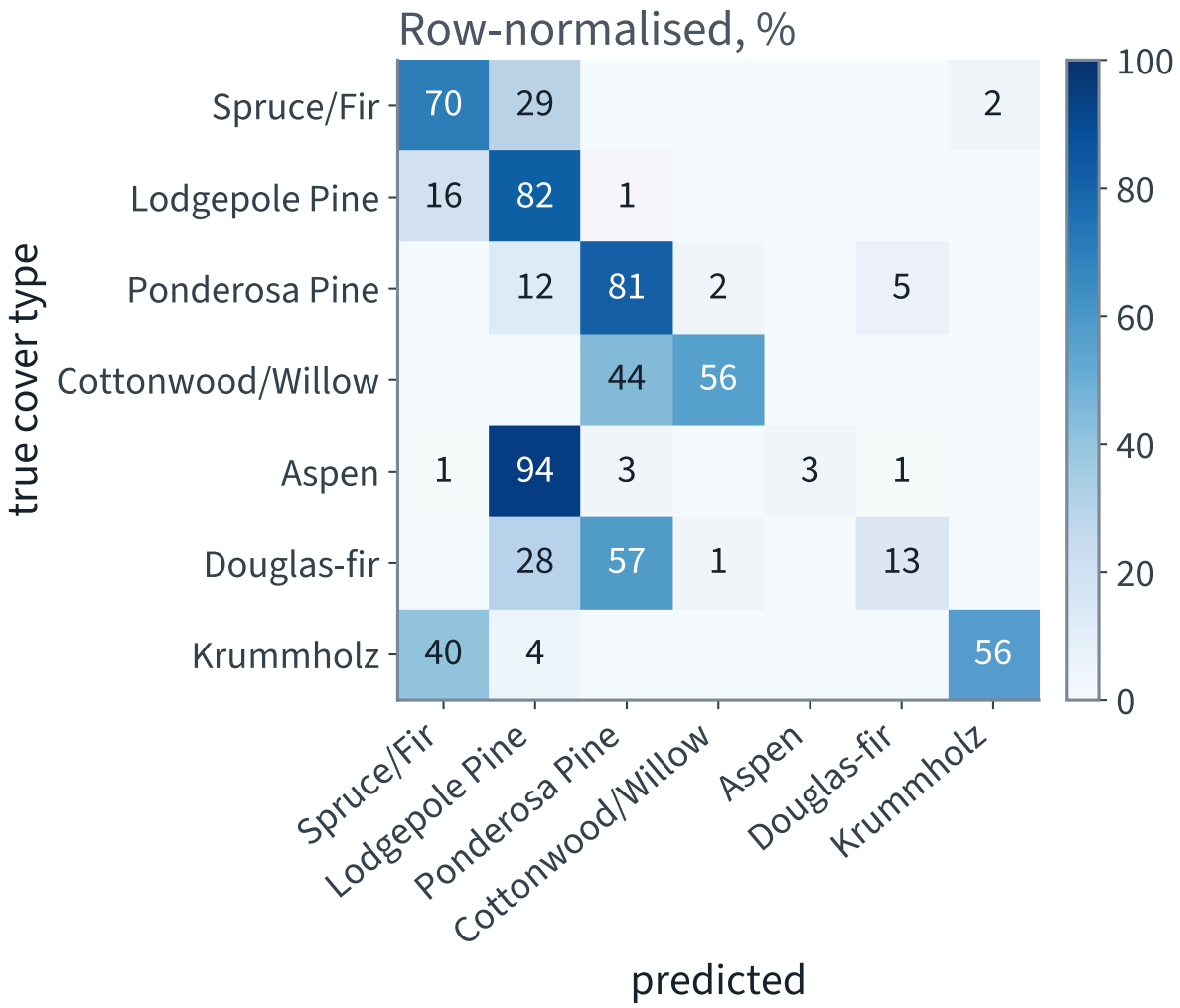
Against a constant baseline of 48.8%, and every choice above was made on cross-validated folds of the training set.

One number for seven species is a summary, not a finding

```
from sklearn.metrics import ConfusionMatrixDisplay, classification_report

print(classification_report(y_test, tree.predict(X_test),
                            target_names=COVER_NAMES, digits=3))
ConfusionMatrixDisplay.from_estimator(tree, X_test, y_test,
                                      normalize="true")
```

`normalize="true"` divides by the row total, so each row reads as “of the patches that really were this species, where did they go?”

Where the errors actually are



✗ **Read the Aspen row. Almost all of it is in the Lodgepole Pine column.**

The same matrix, as one column of numbers

Cover type	Share of the data	Recall
Lodgepole Pine	48.8%	**82.0% ✓**
Ponderosa Pine	6.2%	**80.8% ✓**
Spruce/Fir	36.5%	69.6%
Krummholz	3.5%	56.4%
Cottonwood/Willow	0.5%	56.1%
Douglas-fir	3.0%	**✗ 12.8%**
Aspen	1.6%	**✗ 2.6%**

✗ The model finds fewer than 3 Aspen patches in every hundred. On the headline number that costs 1.6 points and is invisible.

Why the tree cannot see Aspen

FAILURE CONDITION · RARE CLASSES UNDER A WEIGHTED IMPURITY

J weights each child's impurity by *how many instances it holds*. A split that isolates a class at 1.6% of the node moves the weighted average by almost nothing, so the greedy search never chooses one.

- With `max_depth=8` there are at most 256 leaves to allocate across seven species, and the algorithm spends them where the instances are
- This is Lecture 3's lesson in a new costume: an objective that averages over instances rewards ignoring the rare class, and here the model obliges on two species of seven

It is a property of the objective in Step 6, not a bug in the implementation. Read Step 6 again and you could have predicted it.

What reweighting buys, and what it costs

`class_weight="balanced"` multiplies each instance's contribution to J by the inverse of its class frequency, so every class carries the same total weight.

- Aspen recall rises from 2.6% to most of the class — the split that isolates it is now worth choosing
- Overall accuracy falls sharply, because the two species that are 85% of the ground now carry 2/7 of the weight instead of 85% of it
- The notebook fits both and prints the pair. Predict the direction of each before you run it

✓ A different model answering a different question. Which one the agency wants is not a machine learning decision — but quoting either number without the other is an argument rather than a measurement.

What goes in the report

ONE SENTENCE THE AGENCY NEEDS

> “This map is 73.0% accurate overall, but it identifies fewer than three Aspen stands in a hundred. Do not use it to decide anything about Aspen or Douglas-fir.”

A model can be fit for one purpose and unfit for another, in the same deployment. Saying which is which is the job.

This is Lecture 2’s error slice applied to classes rather than to districts. Same idea, same obligation.

THE LAST FIFTEEN MINUTES

Regression trees, and two ways a tree breaks

15 minutes · examinable

The same algorithm, for regression

`DecisionTreeRegressor` is CART with the impurity of Step 6 replaced by the mean squared error of the node:

$$J(k, t_k) = \frac{m_{\mathrm{L}}}{m}\,\mathrm{MSE}_{\mathrm{L}} + \frac{m_{\mathrm{R}}}{m}\,\mathrm{MSE}_{\mathrm{R}}$$

where a node's MSE is measured about its own mean, $\frac{1}{m}\sum_i (y^{(i)} - \bar{y})^2$.

A leaf predicts the **mean** of the training targets that reached it, so the output is a step function — piecewise constant, one step per leaf.

- An unconstrained regression tree drives its *training* error to exactly zero: enough leaves, and every training row gets its own step
- A tree cannot **extrapolate**. Beyond the range of the training targets it returns the nearest leaf's mean, forever — which is the thing to know before using one on a trending series
- The same hyperparameters regularise it, and for the same reason

Failure condition 1 · every boundary is axis-aligned

Every split is of the form $x_k \leq t_k$, so every decision boundary is **perpendicular to an axis**.

WHEN IT BITES

A boundary that genuinely runs at 45° has to be approximated by a staircase of axis-aligned steps, and each step costs a level of depth. Rotate the data by 45° and the same problem needs a much deeper tree, even though nothing about the problem changed.

- Scaling a column is harmless — monotone in that column alone
- *Rotating* the space is not: it mixes columns, and the family of candidate splits is not rotation-invariant
- Géron's remedy is to apply PCA first, which tends to orient the axes better. That is Chapter 7, in Lecture 8

Failure condition 2 · ask a question the accuracy cannot answer

You have a model whose every prediction comes with a rule a person can read.

Fit it again — same hyperparameters, same seed — on *almost* the same training set. Ninety per cent of the same rows.

✗ How similar are the two sets of rules?

Almost everybody expects “very”. The answer has two halves and they point in opposite directions.

The experiment

```
trees, preds = [], []
for seed in range(20):
    Xs, _, ys, _ = train_test_split(X_train, y_train, train_size=0.9,
                                    stratify=y_train, random_state=1000 + seed)
    t = DecisionTreeClassifier(max_depth=8, min_samples_leaf=20,
                               random_state=42).fit(Xs, ys)
    trees.append(t)
    preds.append(t.predict(X_test))       # the same 12,000 test patches
```

Twenty trees. Each sees 90% of the same training set, so any two of them share about 80% of their rows. Everything else is identical — including `random_state`.

The accuracy does not move. The predictions do.

Over the 20 refits	Measured
Accuracy, mean	72.99%
Accuracy, standard deviation	**0.25% ✓**
Accuracy, range	72.39% – 73.38%
Pairwise prediction disagreement, over all 190 pairs	**✗ 9.1%**
… worst pair	**✗ 15.4%**
Test patches all twenty trees agree on	**✗ 71.4%**

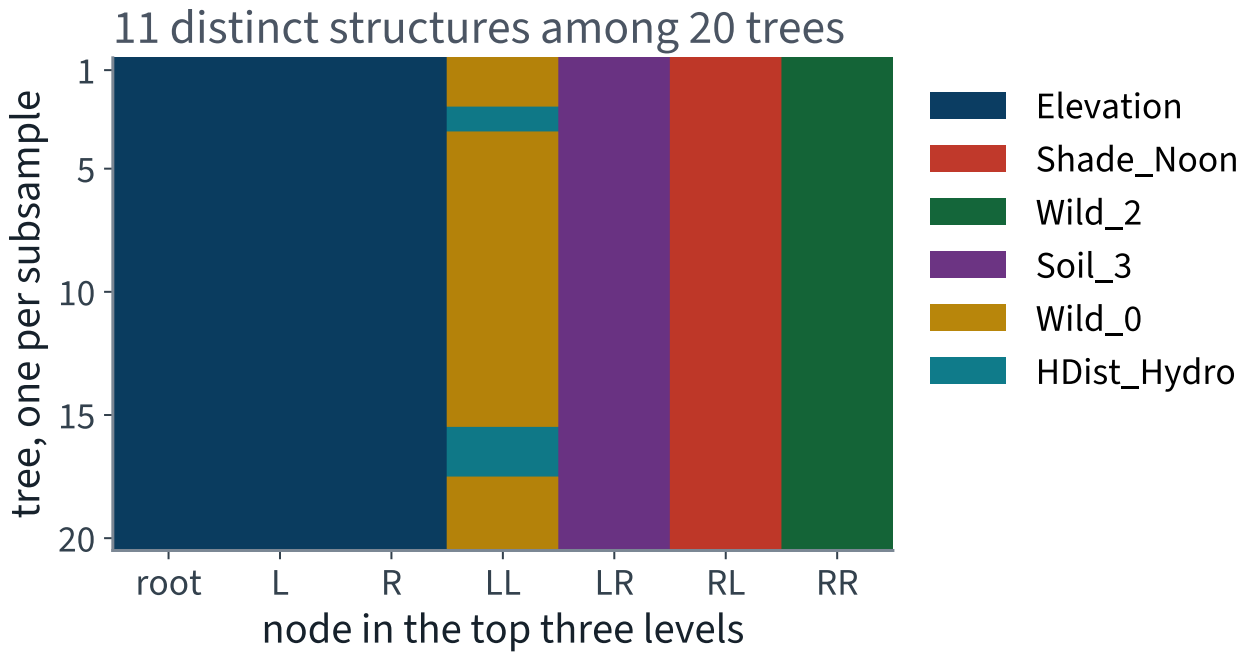
✗ Two trees with the same accuracy disagree about the species of one patch in eleven.

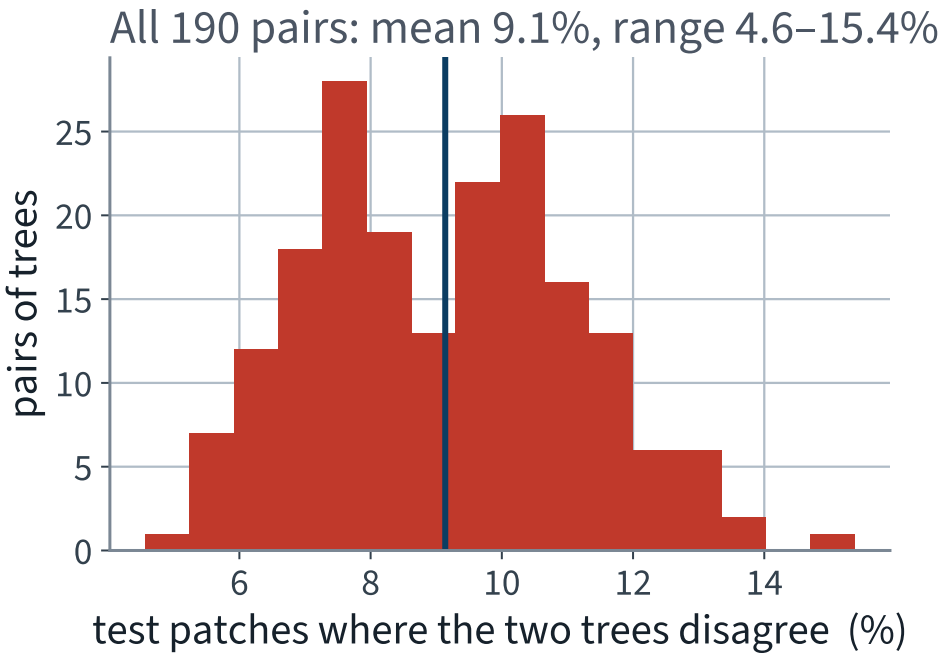
Where in the tree does it change?

Top of the tree	Across the 20 refits
Feature at the root	**Elevation, 20 times out of 20 ✓**
Distinct thresholds at the root	4, spanning 15 metres of elevation
Distinct structures in the top three levels	11 of 20
Leaves in the whole tree	151 to 170
Columns the tree consults	22 to 27 of 54

The part you put on a slide is stable. The part that decides is not.

Twenty trees, one training set





✗ Left: the questions barely move. Right: one prediction in eleven changes anyway.

Why those two results are consistent

- A tree is a **hierarchy**. The root split is chosen from 48,000 patches and wins by a wide margin, so it is stable — which is also why Gini and entropy agreed about it 20 times out of 20
- A node eight levels down was chosen from a few hundred patches, where two candidate splits are often separated by a hair
- Change one row and the loser becomes the winner — and *everything below that node is now a different tree*
- Accuracy is an average over 12,000 patches, and averages hide substitutions: a tree that loses a patch here and gains one there scores the same

✗ A stable metric is not evidence of a stable model. It is evidence that you measured the wrong thing.

Name what we are looking at

FAILURE CONDITION · THIS IS VARIANCE

In the sense of Lecture 5's decomposition — the sensitivity of the fitted model to the particular training sample. Trees have it badly, for a structural reason: the greedy, hierarchical fit turns an arbitrarily small change in one split into an arbitrarily large change in the subtree below it. Nothing smooths it out.

Failure condition 1 was the same fact in a different hat. Rotating the data changes which axis-aligned split wins by a hair, and the tree below it changes completely.

✓ The next lecture derives what averaging does to variance, and builds the models that exploit it. It also shows what the repair costs: the justification we spent this lecture earning.

Scope, and what we deliberately did not do

Examinable: the impurity derivation in full — the two conditions, both formulas, the maxima, the CART objective, $\Delta I \geq 0$, and why the two criteria usually agree. Reading a node and a root-to-leaf path. What each regularisation hyperparameter controls. Both failure conditions, stated as properties.

- We scaled nothing — splits are invariant under monotone transformations of a single feature
- We encoded nothing — wilderness area and soil type arrived one-hot, and a tree splits a 0/1 column at 0.5, reading as “is it soil type 31?”
- We touched the test set once. Every choice was made on cross-validated folds of the training set

Graphviz, Colab mechanics, wall-clock timings, the internals of `fetch_covtype`.

NOT EXAMINABLE — ENGINEERING

The notebook for this lecture

Open Lecture 6 in Colab

- **Runs on free CPU.** The download is about 11 MB; the slowest cells are the depth sweep and the 2-D grid, a couple of minutes each on two cores
- Everything on today's slides, computed on the same rows with the same seed: the split, the baseline, both impurity curves, the paired criterion comparison, the sweep, the grid, the shipped tree, the traced prediction, the confusion matrix and the stability experiment
- It checks the mathematics numerically — that $\Delta I \geq 0$ at every node of the fitted tree, and that both maxima are attained where Step 5 says

> **WHAT TO CHANGE BEFORE THE NEXT LECTURE**
>
> Raise `max_depth` from 8 to 12 and re-run the trace. Accuracy rises by about four points, the justification roughly doubles in length, and the pair is the trade this whole lecture is about.

Every notebook in the course

01 · What machine learning is, and how we will work

02 · The end-to-end project

03 · Classification and its metrics

04 · Training models

05 · Regularisation and the bias–variance trade-off

06 · Decision trees

07 · Ensembles and random forests

08 · Dimensionality reduction and unsupervised learning

09 · Neural networks, from the perceptron up

10 · PyTorch

11 · Training deep networks

12 · Convolutional networks

13 · Transfer learning

14 · Detection and segmentation

15 · Time series

16 · Recurrent networks

17 · Text

18 · Attention and transformers

19 · Information retrieval: the lexical foundation

20 · Information retrieval: dense retrieval

21 · Recommender systems: from ratings to factors

22 · Recommender systems: neural, and evaluated honestly

23 · Vision transformers and multimodal retrieval

24 · Generation, retrieval-augmented systems, and where this leaves you

What we did

1. Chose the model family from the *shape* of the answer the agency needed — a per-prediction justification in at most eight conditions — before looking at the data
2. Derived impurity from one property, concavity of φ: Gini is the probability that two draws from a node differ, entropy is the bits needed to transmit a class, both are zero only at purity and maximal only at uniformity
3. Wrote down what CART minimises — $J = \frac{m_{\mathrm{L}}}{m} I_{\mathrm{L}} + \frac{m_{\mathrm{R}}}{m} I_{\mathrm{R}}$ — and proved from concavity that a split never increases impurity
4. Showed that both criteria rank comparable splits identically, then measured it: the same root split 20 times out of 20, and 0.44 points of accuracy between them. Real, and not worth tuning
5. Grew a tree, priced the eight-condition requirement at 8.0 points of accuracy, and traced one prediction to eight readable conditions and a leaf of 463 patches
6. 73.0% overall and 2.6% recall on Aspen — a consequence of the weighted average in J, not a bug — so never ship the first number without the second
7. Named two failure conditions: boundaries are axis-aligned, and the fitted tree is unstable — 9.1% of predictions move when 10% of the rows do, while the accuracy moves by 0.25 points

In the book

Topic	Chapter
Training, visualising and reading a decision tree	5
Gini impurity, entropy, and choosing between them	5
The CART cost function and greediness	5
Computational complexity of trees	5
Regularisation hyperparameters; nonparametric models	5
Regression trees; axis sensitivity; instability	5
Multiclass classification, confusion matrices, recall	3 (Lecture 3)
Cross-validation and grid search	2 (Lecture 2)

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 6

five, in the style of the written paper

Exercises — Lecture 6

1 Gini and entropy give nearly the same trees on CoverType. State the property they share that explains it, and name a case where they can differ. [4]

2 A decision tree is described as ‘nonparametric’. Explain what that means here, and what it implies about the need for hyperparameters. [4]

3 The brief allows at most eight conditions per decision. You measure the number of conditions as `decision_path().sum() - 1`. Explain the -1. [3]

Solutions on Lecture 7’s deck.

Exercises — Lecture 6 (continued)

4 Refitting a tree on two halves of the same data gives visibly different trees. Does this make the model unreliable? Answer, with a reason. [5]

5 Per-class recall shows one class far below the rest. Give two responses that do not involve changing the model, and say which you would try first. [4]

Solutions on Lecture 7’s deck.

THE FIVE SET AT THE END OF LECTURE 5

Solutions Lecture 5

not part of this lecture

Solutions — Lecture 5

1. Write the bias–variance decomposition of the expected squared error, and name the one term no model can…

✓ $\mathbb{E}[(y - \hat{f})^2] = \text{bias}^2 + \text{variance} + \sigma^2$; the noise σ^2 cannot be reduced.

- Two passengers with identical features and different outcomes put a floor under any predictor
- A model reporting error below that floor has been measured wrongly

2. A learning curve shows training and validation error both high and close together, and flat as data is added.…

✓ High bias. More data would not help.

- The two curves have already met, so the gap — the variance — is small
- The remaining error is the model's inability to represent the function, which more rows do not change

Solutions — Lecture 5 (2)

3. Ridge adds αI to $X^\top X$ before inverting. Give the numerical consequence, in terms of the…

✓ It raises every eigenvalue by α, so the condition number falls and the inverse is stable.

- Near-collinear columns give eigenvalues near zero, which the inverse amplifies
- The bound on the solution's sensitivity improves as α grows — at the cost of bias

4. Why must features be scaled before ridge or lasso, when they need not be for ordinary least squares?

✓ The penalty is on the coefficients, and a coefficient's size depends on its feature's units.

- Least squares is equivariant to rescaling a column: the coefficient absorbs it
- A penalised fit is not, so an unscaled feature is penalised in proportion to the units someone chose

Solutions — Lecture 5 (3)

5. You tune α over a grid using cross-validation, then report the best cross-validated score as your…

✓ The best-of-grid score is optimistic: it is a minimum over the same folds used to choose.

- Selection over k candidates on the same data biases the winner downward in error
- The honest number comes from data that took part in no choice — a held-out test set, scored once

LECTURE 7 · GÉRON CH. 6

Ensembles and random forests

The variance of an average of correlated predictors, derived — one result that explains bagging, feature subsampling and random thresholds

Out-of-bag evaluation, feature importance, boosting and stacking

And what averaging costs: the justification you spent today earning

Run the Lecture 6 notebook first